

Modèle relationnel

Modèle relationnel complet de la base de données (notation MERISE), généré à partir du schéma Prisma réellement en production sur ce projet — pas un modèle théorique reconstitué a posteriori.

Projet : next-one (Power Delivery)

Source : prisma/schema.prisma

Date : 25 juillet 2026

Préparé pour : Basma Boutaib

Sommaire

1. Vue d'ensemble (diagramme entités-associations)

2. Notation utilisée

3. Domaine "Utilisateurs et marchands"

4. Domaine "Cœur métier — colis"

5. Domaine "Ramassage"

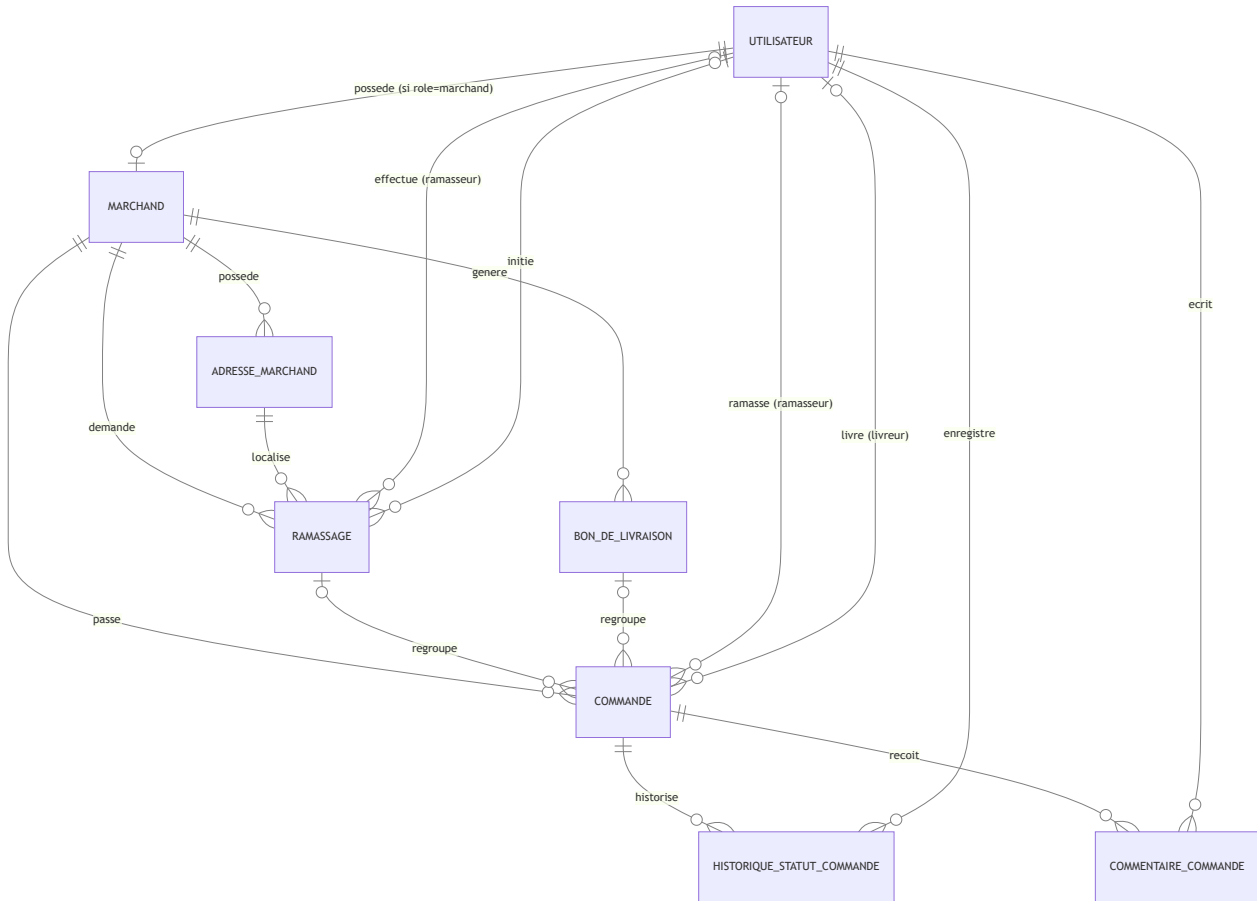
6. Modules indépendants

7. Tableau récapitulatif des cardinalités

8. Règles de gestion associées au modèle

1. Vue d'ensemble

11 tables au total : 3 pour les comptes (utilisateurs, marchands, adresses), 4 pour le cycle de vie d'un colis, 1 pour le ramassage, 2 modules indépendants (liste noire, produits — pas encore reliés au reste).



Lecture des cardinalités Mermaid : `||` = exactement un (1,1) • `|o` = zéro ou un (0,1) • `o{` = zéro ou plusieurs (0,n)

- `}|` = un ou plusieurs (1,n).

2. Notation utilisée

Chaque table est présentée selon la notation relationnelle MERISE classique :

NOM_TABLE

(id, attribut1, attribut2, #cle_etrangere, ...)

- **Souligné** = clé primaire
- **##**préfixe = clé étrangère (référence une autre table)
- *Italique* = contrainte d'unicité (autre que la clé primaire)

Les noms de colonnes correspondent exactement aux noms physiques en base PostgreSQL (tels que définis par les annotations `@map` du schéma Prisma), pas aux noms de variables TypeScript utilisés dans le code de l'application.

3. Domaine "Utilisateurs et marchands"

utilisateurs

(id, nom_complet, telephone, email, mot_de_passe_hash, role, actif, date_creation, derniere_connexion, reset_token_hash, reset_token_expire)

`role` ∈ {admin, finance, sav, agent_confirmation, marchand, livreur, ramasseur}. Une seule table pour tous les rôles : c'est ce champ qui distingue un compte admin d'un compte marchand ou livreur (voir le document sur la sécurité des sessions pour le détail).

marchands

(id, #utilisateur_id (unique), nom_boutique, raison_sociale, ice_rc, type_compte, ville, rib, cin, site_web, adresse, nom_banque, rib_photo_url, ville_ramassage, registre_commerce, statut, ramassage_recurrent_actif, ramassage_jours, ramassage_creneau_horaire, date_creation)

`utilisateur_id` est à la fois clé étrangère **et** unique : chaque marchand correspond à exactement un compte utilisateur, et un utilisateur a au plus un profil marchand (relation 1-1). `type_compte` ∈ {marchand, entreprise, dropshipping}. `statut` ∈ {en_attente_validation, actif, suspendu}.

adresses_marchand

(id, #marchand_id, libelle, adresse_complete, est_par_defaut)

Un marchand peut avoir plusieurs adresses de ramassage enregistrées ; chaque `ramassage` référence l'une d'entre elles.

4. Domaine "Cœur métier — colis"

commandes

(id, code_suivi, #marchand_id, client_nom, client_telephone, ville, adresse, code_postal, produit_description, quantite, poids_kg, montant_cod, notes, statut, etat_paieement, a_risque, #ramasseur_id, #ramassage_id, #livreur_id, #bon_livraison_id, source, date_creation, date_confirmation, date_collecte, date_livraison, gps_livraison_lat, gps_livraison_lng, signature_url, photo_preuve_url, motif_retour, cin_url)

La table centrale du système. `marchand_id` est obligatoire ; `ramasseur_id`, `livreur_id`, `ramassage_id` et `bon_livraison_id` sont tous optionnels (un colis peut exister avant d'être affecté à qui que ce soit). `statut` compte 27 valeurs possibles (cycle de vie complet centre d'appel → livraison, voir `lib/statuts.ts`) et `etat_paieement` (COD) est **indépendant** du statut de livraison — un colis "livré" peut rester "non payé" tant que le marchand n'est pas remboursé.

bons_de_livraison

(id, numero, #marchand_id, nb_colis, montant_total_cod, date_generation)

Regroupe plusieurs colis "en_attente" d'un même marchand en un lot confirmé d'un coup (numéro au format `BL-AAAA-MMJJ-NNN`).

historique_statuts_commande

(id, #commande_id, ancien_statut, nouveau_statut, #utilisateur_id, horodatage)

Journal d'audit : chaque changement de statut d'un colis est tracé avec qui l'a fait et quand.

commentaires_commande

(id, #commande_id, #utilisateur_id, texte, date_creation)

Notes libres internes (centre d'appel), affichées avec l'historique dans la vue "circuit" du colis côté admin.

5. Domaine "Ramassage"

ramassages

(id, #marchand_id, #adresse_id, date_prevue, creneau_horaire, mode_creation, #initie_par, nb_colis_estimes, nb_colis_reels, #ramasseur_id, statut)

Une tournée de ramassage planifiée chez un marchand. `mode_creation` ∈ {recurrent, manuel} : une tournée récurrente n'a pas d' `initie_par` (générée automatiquement par le planificateur), alors qu'une tournée manuelle trace qui l'a déclenchée (marchand ou admin). Une même tournée peut regrouper plusieurs `commandes` .

6. Modules indépendants

Ces deux tables existent dans le schéma mais ne sont reliées à aucune autre table pour l'instant — ce sont des modules autonomes, prévus pour une intégration future.

liste_noire

(id, type, valeur, nom_associe, motif, niveau_risque, date_creation)

`type` ∈ {telephone, client, adresse} — permet de blacklister un numéro, un client ou une adresse. `motif` ∈ {fraude, refus_repetitifs, faux_numero, client_agressif, impaye, autre}. `niveau_risque` ∈ {moyen, eleve, critique}.

produits

(id, nom, sku, stock, prix_unitaire, seuil_alerte, date_creation)

Catalogue produit avec suivi de stock — pas encore relié aux commandes (le champ `produit_description` de `commandes` reste aujourd'hui un texte libre, pas une référence vers cette table).

7. Tableau récapitulatif des cardinalités

Table source	Table cible	Cardinalité	Clé étrangère	Obligatoire ?
utilisateurs	marchands	1,1 — 0,1	marchands.utilisateur_id	Oui côté marchand
marchands	adresses_marchand	1,1 — 0,n	adresses_marchand.marchand_id	Oui
marchands	commandes	1,1 — 0,n	commandes.marchand_id	Oui
marchands	ramassages	1,1 — 0,n	ramassages.marchand_id	Oui
marchands	bons_de_livraison	1,1 — 0,n	bons_de_livraison.marchand_id	Oui
adresses_marchand	ramassages	1,1 — 0,n	ramassages.adresse_id	Oui
ramassages	commandes	0,1 — 0,n	commandes.ramassage_id	Non
bons_de_livraison	commandes	0,1 — 0,n	commandes.bon_livraison_id	Non
utilisateurs (ramasseur)	commandes	0,1 — 0,n	commandes.ramasseur_id	Non
utilisateurs (livreur)	commandes	0,1 — 0,n	commandes.livreur_id	Non
utilisateurs (ramasseur)	ramassages	0,1 — 0,n	ramassages.ramasseur_id	Non
utilisateurs (initiateur)	ramassages	0,1 — 0,n	ramassages.initie_par	Non
commandes	historique_statuts_commande	1,1 — 0,n	historique_statuts_commande.commande_id	Oui
utilisateurs	historique_statuts_commande	1,1 — 0,n	historique_statuts_commande.utilisateur_id	Oui
commandes	commentaires_commande	1,1 — 0,n	commentaires_commande.commande_id	Oui
utilisateurs	commentaires_commande	1,1 — 0,n	commentaires_commande.utilisateur_id	Oui

8. Règles de gestion associées au modèle

Règle	Traduction dans le modèle
RF-22 — un marchand auto-inscrit reste bloqué tant qu'un admin ne l'a pas validé	<code>marchands.statut = en_attente_validation</code> par défaut, synchronisé avec <code>utilisateurs.actif = false</code> jusqu'à approbation
RG-15 / RG-16 — modèle hybride de ramassage (récurrent + manuel)	<code>ramassages.mode_creation</code> + <code>ramassages.initie_par</code> nullable pour le mode récurrent
État de paiement COD indépendant du statut de livraison	Deux colonnes séparées sur <code>commandes</code> : <code>statut</code> et <code>etat_paiement</code>
Un colis n'est affecté à personne au départ	<code>ramasseur_id</code> , <code>livreur_id</code> , <code>ramassage_id</code> , <code>bon_livraison_id</code> tous nullable sur <code>commandes</code>
Traçabilité complète de chaque changement de statut	Table <code>historique_statuts_commande</code> dédiée, jamais de mise à jour destructive du statut sans trace